

RAG-Governor: Unified Governance for Trustworthy and Cost-Efficient Retrieval-Augmented Generation with SLO Guarantees

Wuyang Zhang*
Northeastern University
Boston, MA, USA
zhang.noc@northeastern.edu

Zhen Luo
Northeastern University
Boston, MA, USA
luo.zhe@northeastern.edu

Yebo Cao
Carnegie Mellon University
Pittsburgh, PA, USA
yeboc@alumni.cmu.edu

Jianming Ma
Northeastern University
Boston, MA, USA
ma.jianming@northeastern.edu

Yinzhi Jin
Carnegie Mellon University
Pittsburgh, PA, USA
yinzhi@alumni.cmu.edu

Wangming Yuan
George Mason University
Fairfax, VA, USA
yuan.wangming@gmu.edu

Abstract

Retrieval-Augmented Generation (RAG) grounds language model outputs in external knowledge to reduce hallucinations, yet current architectures treat retrieval as a one-time preprocessing step that cannot detect errors during generation, adapt to varying source reliability, or optimize costs under latency constraints. We present RAG-Governor, a unified governance layer that reframes RAG as continuous monitoring and adaptation. The system integrates three modules: a validation engine that performs real-time entity extraction and entropy-based uncertainty monitoring to intercept hallucinations during token generation, a source manager that maintains dynamic trust scores through cross-verification feedback loops, and a hierarchical memory system combining knowledge graphs with semantic caching to reduce retrieval costs while ensuring information freshness.

Evaluation across four benchmarks spanning factual and temporal question answering demonstrates that RAG-Governor reduces hallucination rates by up to 42% compared to reliability-aware baselines while decreasing retrieval costs by 40–52% through intelligent cache utilization. The system achieves 56% higher throughput than standard RAG and maintains SLO violation rates below 20% under strict latency budgets where existing methods exceed 45%. These results establish continuous governance as a practical paradigm for deploying trustworthy and cost-efficient RAG systems in production environments.

CCS Concepts

• **Computing methodologies** → **Natural language processing; Neural networks**; • **Information systems** → **Information retrieval**; • **Computer systems organization** → *Cloud computing*.

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License.
DMIT 2026, Beijing, China
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2172-4/26/02
<https://doi.org/10.1145/3808707.3808839>

Keywords

Retrieval-Augmented Generation, Large Language Models, Hallucination Detection, Source Trust Management, Knowledge Graph, Semantic Caching, SLO Optimization, LLM Serving

ACM Reference Format:

Wuyang Zhang, Yebo Cao, Yinzhi Jin, Zhen Luo, Jianming Ma, and Wangming Yuan. 2026. RAG-Governor: Unified Governance for Trustworthy and Cost-Efficient Retrieval-Augmented Generation with SLO Guarantees. In *The 2nd International Conference on Digital Management and Information Technology (DMIT 2026)*, February 06–08, 2026, Beijing, China. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3808707.3808839>

1 Introduction

Large language models have transformed information access, yet their tendency to hallucinate remains a critical deployment barrier. This challenge intensifies for time-sensitive queries where facts evolve and for topics with conflicting sources. In enterprise settings, such errors carry significant consequences: legal exposure, financial losses, and reputational damage.

Retrieval-Augmented Generation (RAG)^[15] addresses this by grounding outputs in external knowledge retrieved before generation. However, current architectures treat retrieval as a preprocessing step that cannot intercept hallucinations during generation, assume all sources are equally reliable, trigger full retrieval even when unnecessary, and introduce latencies that violate service level objectives. These limitations share a root cause: the assumption that retrieval is a one-time operation rather than an ongoing process.

We propose treating RAG as *continuous governance*: constantly validating generated content against trusted knowledge, assessing source reliability based on observed performance, and adapting retrieval decisions in real time. This paradigm enables proactive interception of hallucinations during generation rather than post-hoc correction.

RAG-Governor implements this paradigm through three integrated modules. The *Validation Engine* monitors token generation for uncertainty signals and triggers targeted retrieval when confidence drops. The *Source Manager* maintains dynamic trust scores based on latency, accuracy, and ranking performance. The *Memory System* combines a knowledge graph with hierarchical caching to

serve queries without external retrieval when possible. The system integrates non-intrusively with inference engines like vLLM^[13].

This paper makes the following contributions:

- A real-time validation engine that intercepts hallucinations during generation, reducing hallucination rates by up to 42% over reliability-aware baselines.
- A trust-aware source manager with dynamic scoring and feedback loops that routes queries to reliable providers.
- A hierarchical memory system combining knowledge graphs and semantic caching that reduces retrieval costs by 40–52%.
- Comprehensive evaluation on four benchmarks demonstrating improvements in quality, throughput, and SLO compliance.

2 Related Work

We review three areas: RAG quality and hallucination detection, trust management and knowledge infrastructure, and SLO-aware serving.

2.1 RAG Quality and Hallucination Detection

Lewis et al.^[15] introduced RAG to ground outputs in retrieved knowledge, yet hallucinations persist. Recent approaches include hallucination-aware tuning^[17], self-corrective retrieval^[1], and mechanistic analysis of context-knowledge balance^[18]. Uncertainty quantification offers principled detection: semantic entropy^[7] captures meaning-level confidence, while hidden-state probes^[11] reduce computational costs. These methods operate post-hoc rather than intercepting errors during generation.

2.2 Trust Management and Knowledge Infrastructure

Source quality varies, yet most RAG systems treat sources equally. RA-RAG^[9] estimates reliability through cross-verification; Pistis-RAG^[3] aligns ranking with LLM preferences. For efficiency, GraphRAG^[6] constructs entity knowledge graphs for global reasoning, while RAGCache^[10] achieves 4x TTFT reduction through hierarchical caching. These systems lack integration between trust assessment and hallucination detection.

2.3 SLO-Aware LLM Serving

Production RAG requires efficient infrastructure. vLLM^[13] eliminates memory fragmentation for 24x throughput gains; SOLA^[8] improves SLO attainment to 99.4% through state-aware scheduling. These focus on infrastructure rather than RAG-specific latency optimization.

2.4 Positioning

RAG-Governor uniquely integrates real-time hallucination interception, dynamic trust management with feedback loops, hierarchical caching, and SLO-aware retrieval into a unified governance layer for production RAG.

3 System Design

We present RAG-Governor, a unified governance layer that operates on top of existing LLM inference engines to provide real-time

Algorithm 1 RAG Interception Protocol

Require: Tokens $\mathbf{x}$, thresholds θ_H, θ_K

```

1:  $\mathcal{E} \leftarrow \text{EXTRACTENTITIES}(\mathbf{x})$ 
2: for each entity  $e \in \mathcal{E}$  do
3:    $H_e \leftarrow \text{COMPUTEENTROPY}(e, \mathbf{x})$ 
4:    $s_e \leftarrow \text{KBSIMILARITY}(e)$ 
5:   if  $H_e > \theta_H$  or  $s_e < \theta_K$  then
6:      $\text{PAUSEGENERATION}()$ 
7:      $\mathbf{d} \leftarrow \text{RETRIEVE}(e, \mathbf{x})$ 
8:     if  $\text{VERIFY}(\mathbf{d}, e) = \text{inconsistent}$  then
9:        $\mathbf{x} \leftarrow \text{AMENDCONTEXT}(\mathbf{x}, \mathbf{d})$ 
10:    end if
11:     $\text{RESUMEGENERATION}(\mathbf{x})$ 
12:   end if
13: end for
```

hallucination detection, trust-aware source management, and cost-efficient memory hierarchies. This section describes the system architecture and its three core modules.

3.1 System Architecture

RAG-Governor integrates with production inference engines such as vLLM^[13] and HuggingFace Transformers through a non-intrusive middleware layer. Rather than modifying the underlying inference code, our system intercepts token generation callbacks and retrieval requests through well-defined hooks, enabling deployment alongside existing infrastructure without architectural changes.

Three modules operate in a tightly coupled feedback loop. When the *Validation Engine* detects uncertainty during generation, it queries the *Source Manager* for verification from trusted sources; successful verifications then flow into the *Memory System*, which caches results and updates the knowledge graph. This bidirectional data flow ensures that each module’s decisions inform the others: interception outcomes update trust scores, verified retrievals enrich cached knowledge, and cache hits reduce the validation engine’s need to trigger external verification.

When a query arrives, the Memory System first attempts to satisfy it from cached or graph-based knowledge. If generation is required, the Validation Engine monitors each token for uncertainty signals, triggering interception and Source Manager verification when confidence drops. Performance metrics from each interaction feed back into trust scores and cache policies.

3.2 Real-time Semantic Validation

The Validation Engine operates parallel to token generation, performing lightweight entity extraction and uncertainty estimation within the latency constraints of autoregressive decoding.

Streaming Entity Extraction. We employ a DistilBERT-based NER model^[19] with 66M parameters, retaining 97% of BERT’s accuracy at 60% faster inference. The model identifies named entities, temporal expressions, numerical facts, and domain-specific claims on a sliding window of generated tokens, completing within the 10–50ms budget between token samples.

Entropy-based Hallucination Detection. Following Farquhar et al. [7], we monitor semantic entropy rather than token-level uncertainty to capture meaning-level confidence. We approximate this efficiently using probes on hidden states^[11]. Let $H(t)$ denote the entropy at token position t :

$$H(t) = - \sum_{v \in \mathcal{V}} p(v \mid \mathbf{x}_{<t}) \log p(v \mid \mathbf{x}_{<t}) \quad (1)$$

where $\mathcal{V}$ is the vocabulary and $\mathbf{x}_{<t}$ represents the context. A hallucination signal is raised when $H(t) > \theta_H$ for tokens within an entity span, or when extracted entities exhibit low similarity to knowledge graph entries.

Interception Protocol. Algorithm 1 details the hallucination handling procedure. Upon detecting a signal, the system pauses generation, queries the Source Manager for verification, and injects corrected context if retrieval confirms an inconsistency. Each interception updates source trust scores based on outcome.

3.3 Trust-Aware Source Management

The Source Manager maintains dynamic trust assessments for retrieval sources, adapting query routing based on observed reliability and SLO constraints.

Multi-Factor Trust Scoring. Following reliability-aware retrieval^[9], we compute a composite trust score $T(s)$ for each source s :

$$T(s) = \alpha \cdot \text{SLO}(s) + \beta \cdot \text{Acc}(s) + \gamma \cdot \text{Rank}(s) \quad (2)$$

where $\text{SLO}(s)$ measures latency compliance, $\text{Acc}(s)$ captures verification accuracy, and $\text{Rank}(s)$ reflects reranker^[5] placement frequency. The weights sum to one and can be tuned for deployment priorities.

Cross-Source Verification. For claim verification, we retrieve from top- k trusted sources and apply weighted majority voting. Let $v_i(c) \in \{0, 1\}$ indicate whether source s_i supports claim c :

$$\text{Consensus}(c) = \frac{\sum_{i=1}^k T(s_i) \cdot v_i(c)}{\sum_{i=1}^k T(s_i)} \quad (3)$$

Claims with consensus below a threshold are flagged for correction, and sources whose assertions diverge from consensus receive accuracy penalties.

Dynamic Ranking Updates. Trust scores evolve through exponential moving average updates after each retrieval:

$$T(s)_{\text{new}} = \lambda \cdot T(s)_{\text{observed}} + (1 - \lambda) \cdot T(s)_{\text{old}} \quad (4)$$

where $\lambda \in [0.1, 0.3]$ controls adaptation speed. Sources falling below a threshold receive fewer queries, while new sources start neutral and undergo burn-in evaluation.

Adaptive Retrieval Decisions. Query complexity determines retrieval effort. Strong cache hits allow skipping external retrieval, whereas low-confidence queries involving critical facts trigger multi-source verification up to the SLO budget. The Source Manager selects 1 to k sources based on available latency headroom.

Algorithm 2 Retrieval Path Selection

Require: Query q , budget B , threshold θ

```

1:  $t_0 \leftarrow \text{CURRENTTIME}()$ 
2:  $r \leftarrow \text{L1LOOKUP}(q)$ 
3: if  $r \neq \emptyset$  then
4:   return  $r$ 
5: end if
6:  $r, \sigma \leftarrow \text{L2SEARCH}(q)$ 
7: if  $\sigma > \theta$  then
8:   return  $r$ 
9: end if
10:  $r, c \leftarrow \text{KGTRAVERSAL}(q)$ 
11: if  $c > \theta$  then
12:   return  $r$ 
13: end if
14:  $C \leftarrow \text{ESTIMATECOST}(q)$ 
15: if  $C < B - (\text{Now}() - t_0)$  then
16:    $S \leftarrow \text{SELECTSOURCES}(q)$ 
17:    $r \leftarrow \text{RETRIEVE}(q, S)$ 
18:    $\text{UPDATECACHE}(q, r)$ 
19: end if
20:
21: return  $r$ 
```

3.4 Hierarchical Memory System

The Memory System reduces retrieval costs through a four-tier cache hierarchy backed by a knowledge graph that accumulates verified information over time.

Knowledge Graph Construction. We construct a knowledge graph from high-trust retrievals using lightweight entity-relation extraction^[14]. Only passages from sources exceeding trust threshold θ_T are incorporated, and freshness metadata enables time-sensitive queries to prioritize recent information.

Semantic Cache Hierarchy. Following semantic caching approaches^[4,10], we organize cached content into four tiers: **L1** provides exact-match hash lookup (<1ms); **L2** performs embedding similarity search via BGE-M3^[5] (~5ms); **L3** traverses the knowledge graph (10–20ms); and **L4** triggers external retrieval (100–500ms). A hybrid LRU/LFU eviction policy with content-based TTL values ensures temporal facts expire quickly while stable knowledge persists.

Cost-Benefit Retrieval Optimization. Algorithm 2 shows retrieval path selection. Given SLO budget B and confidence threshold θ , the system traverses the cache hierarchy and returns early when confidence suffices. External retrieval proceeds only when the estimated quality gain justifies the cost within the remaining budget.

4 Experiments

We evaluate RAG-Governor on four diverse benchmarks to answer the following research questions:

- **RQ1:** Does real-time validation reduce hallucinations compared to post-hoc correction methods?
- **RQ2:** How does trust-aware source management affect response quality?
- **RQ3:** What cost savings does the hierarchical memory system provide?

- **RQ4:** Can the system maintain SLO compliance under varying latency budgets?

4.1 Experimental Setup

Datasets. We evaluate on four benchmarks: **NaturalQuestions**^[12] (3,610 factual questions from Google queries), **HotpotQA**^[22] (7,405 multi-hop reasoning questions), **TempQuestions** (1,271 time-sensitive questions), and **CRAG-Bench**^[21] (4,409 questions across five domains with temporal dynamics).

Baselines. We compare against five methods: **No-RAG** (Llama-3-8B^[16] without retrieval), **Vanilla RAG** (BM25 with top-5 passages), **SELF-RAG**^[2] (self-reflection with critique tokens), **CRAG**^[20] (corrective retrieval with web search fallback), and **RA-RAG**^[9] (reliability-aware cross-verification).

Implementation. RAG-Governor runs on vLLM^[13] with Llama-3-8B. We set entropy threshold $\theta_H = 1.5$, cache size 10K, and trust smoothing $\lambda = 0.2$. The knowledge graph (Neo4j, ~2M entities) is populated from high-trust retrievals. Experiments use 8 NVIDIA A100 GPUs; we report averages over three runs.

Metrics. Quality metrics include *Hallucination Rate* (percentage of responses with false claims) and *Answer F1* (token-level overlap with references). Efficiency metrics include *RAG Cost* (retrievals per query), *Violation Rate* (queries exceeding latency budget), *Throughput* (queries/second), and *TTFT* (time to first token).

4.2 Main Results

Hallucination Reduction (RQ1). RAG-Governor reduces hallucinations to 9.4% on NaturalQuestions (28% relative improvement over RA-RAG’s 13.1%) and 10.8% on TempQuestions (42% reduction from 18.7%). The larger gap on temporal queries demonstrates real-time validation’s ability to intercept outdated facts during generation.

Quality Improvement (RQ2). Trust-aware source management combined with validation yields consistent F1 gains. On TempQuestions, RAG-Governor achieves 75.2% F1 versus 64.9% for RA-RAG. The advantage grows on challenging datasets: the F1 gap over RA-RAG is 5.6 points on NaturalQuestions but 10.3 points on TempQuestions.

Cost Efficiency (RQ3). RAG-Governor averages only 0.48–0.61 retrievals per query, a 40–52% reduction compared to baselines that require one or more retrievals. The hierarchical memory system serves many queries from cache or knowledge graph without external retrieval.

4.3 SLO Performance

SLO Compliance (RQ4). Under a 200ms budget, RAG-Governor achieves 18.7% SLO violation rate versus 45.6% for Vanilla RAG and 65.7% for CRAG. The hierarchical cache reduces tail latencies by serving queries without external retrieval. At 500ms budget, violations drop to 6.4%.

Throughput and Latency. RAG-Governor achieves 38.4 q/s (56% above Vanilla RAG) with average latency 183ms and P99 of 341ms. Cache hits and adaptive retrieval jointly eliminate unnecessary

latency. Median TTFT drops to 45ms versus 82ms for Vanilla RAG, reflecting immediate generation when cache hits occur.

4.4 Ablation Study

We conduct ablation experiments on TempQuestions to understand each component’s contribution. Figure 3 visualizes the results.

Component Contributions. The validation engine provides the largest quality impact: removing it (w/o *Valid.*) increases hallucination rate from 10.8% to 16.4% (+5.6pp) and drops F1 from 75.2% to 69.8%. Trust management contributes moderately; without it, hallucination rate rises to 13.2%. The memory system primarily affects efficiency: removing it doubles RAG cost (0.48 to 1.00) and latency (142ms to 289ms) while only slightly increasing hallucinations to 11.6%.

4.5 Sensitivity and System Dynamics

Parameter Sensitivity. The entropy threshold θ_H controls detection sensitivity: $\theta_H = 0.5$ achieves 8.2% hallucination rate but increases latency to 198ms, while $\theta_H = 2.5$ allows 15.1% hallucinations with 118ms latency. The default $\theta_H = 1.5$ balances quality and efficiency. Cache size shows diminishing returns beyond 10K entries, which achieves 56.3% hit rate with modest memory.

Runtime Behavior. Trust scores diverge over operation: reliable sources rise to 0.88–0.92 while unreliable ones fall below the demotion threshold. The cache requires ~2K queries to reach steady-state (69% hit rate); pre-warming can accelerate convergence.

5 Conclusion

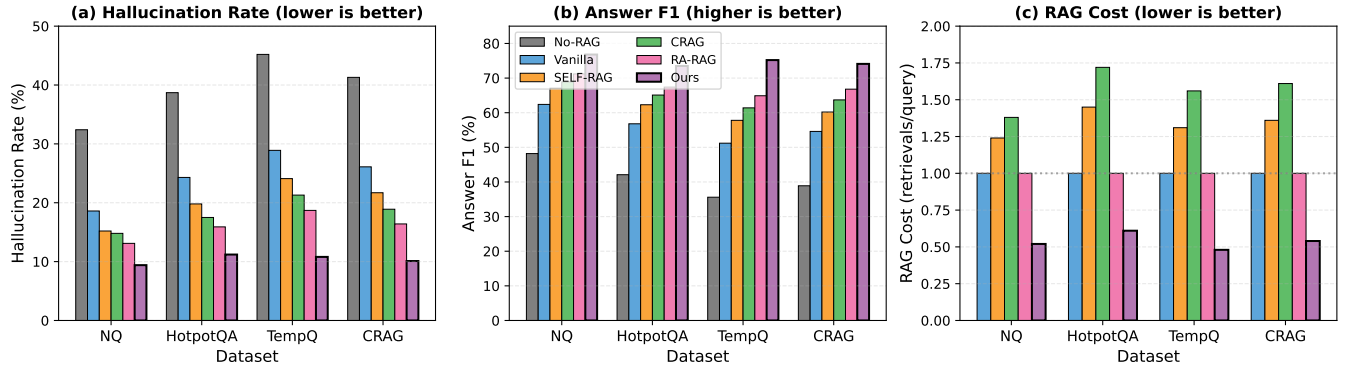
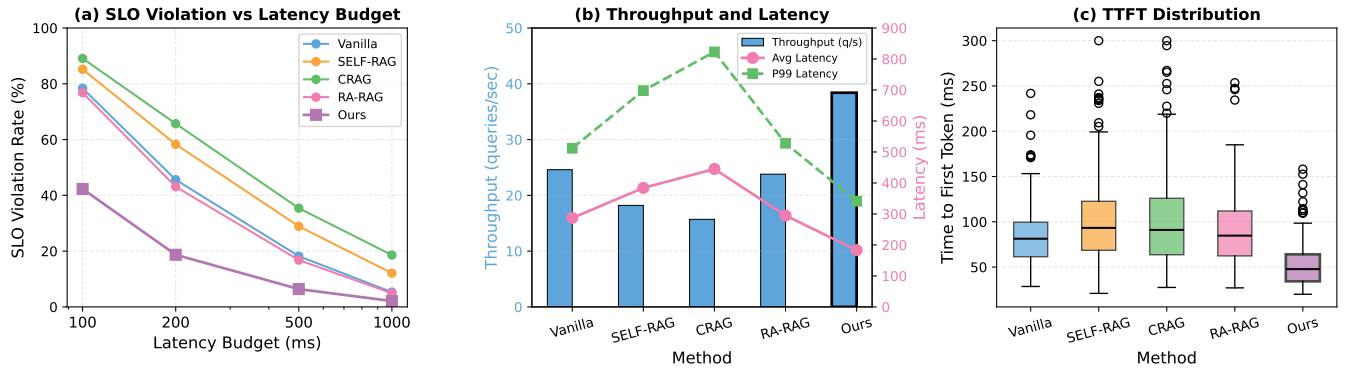
This paper presented RAG-Governor, a unified governance layer that reframes retrieval-augmented generation as continuous monitoring and adaptation rather than one-time preprocessing. By integrating real-time hallucination detection through entropy-based validation, dynamic trust scoring with cross-verification feedback, and hierarchical caching backed by an evolving knowledge graph, the system addresses fundamental limitations of existing RAG architectures. Evaluation across four benchmarks demonstrates substantial improvements: hallucination rates decrease by up to 42% compared to reliability-aware baselines, retrieval costs drop by 40–52% through intelligent cache utilization, and throughput increases by 56% while maintaining SLO violation rates below 20% under strict latency budgets. These results validate the continuous governance paradigm as a practical approach for deploying trustworthy and cost-efficient RAG systems in production environments where both accuracy and latency constraints are critical.

References

- [1] Rishabh Agrawal. 2025. SCMRAG: Self-Corrective Multihop Retrieval Augmented Generation System for LLM Agents. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*. IFAAMAS, 50–58.
- [2] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *Proceedings of the Twelfth International Conference on Learning Representations*.
- [3] Yu Bai, Yanyu Ren, Yingfei Li, Jingjing Xu, Jiawei Zhang, and Lei Li. 2024. Pistis-RAG: A Scalable Cascading Framework Towards Trustworthy Retrieval-Augmented Generation. *arXiv preprint arXiv:2407.00072* (2024).
- [4] Fu Bang and Di Feng. 2023. GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings. In *Proceedings of the*

Table 1: Main results across four benchmarks. Best results in bold, second best underlined. Hall. = Hallucination Rate (% , lower is better), F1 = Answer F1 (% , higher is better), Cost = RAG Cost (retrievals/query, lower is better).

Method	NaturalQuestions			HotpotQA			TempQuestions			CRAG-Bench		
	Hall.	F1	Cost	Hall.	F1	Cost	Hall.	F1	Cost	Hall.	F1	Cost
No-RAG	32.4	48.2	–	38.7	42.1	–	45.2	35.6	–	41.3	38.9	–
Vanilla	18.6	62.4	1.00	24.3	56.8	1.00	28.9	51.2	1.00	26.1	54.6	1.00
SELF-RAG	15.2	67.1	1.24	19.8	62.3	1.45	24.1	57.8	1.31	21.7	60.2	1.36
CRAG	14.8	68.9	1.38	17.5	65.1	1.72	21.3	61.4	1.56	18.9	63.7	1.61
RA-RAG	<u>13.1</u>	<u>71.2</u>	1.00	<u>15.9</u>	<u>67.4</u>	1.00	<u>18.7</u>	<u>64.9</u>	1.00	<u>16.4</u>	<u>66.8</u>	1.00
Ours	9.4	76.8	0.52	11.2	73.5	0.61	10.8	75.2	0.48	10.1	74.1	0.54

**Figure 1: Comparison of hallucination rate, answer F1, and RAG cost across datasets. RAG-Governor achieves the lowest hallucination rates and highest F1 scores while using significantly fewer retrievals than all baselines.****Figure 2: SLO performance comparison. (a) Violation rate decreases as latency budget increases, with RAG-Governor maintaining the lowest violation rate across all budgets. (b) Throughput and latency comparison showing RAG-Governor achieves 56% higher throughput than Vanilla RAG. (c) TTFT distribution demonstrates faster response initiation due to cache hits.**

3rd Workshop on Natural Language Processing Open Source Software (NLP-OSS). Association for Computational Linguistics, 212–218.

- [5] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational

Linguistics, 2308–2328.

- [6] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130* (2024).

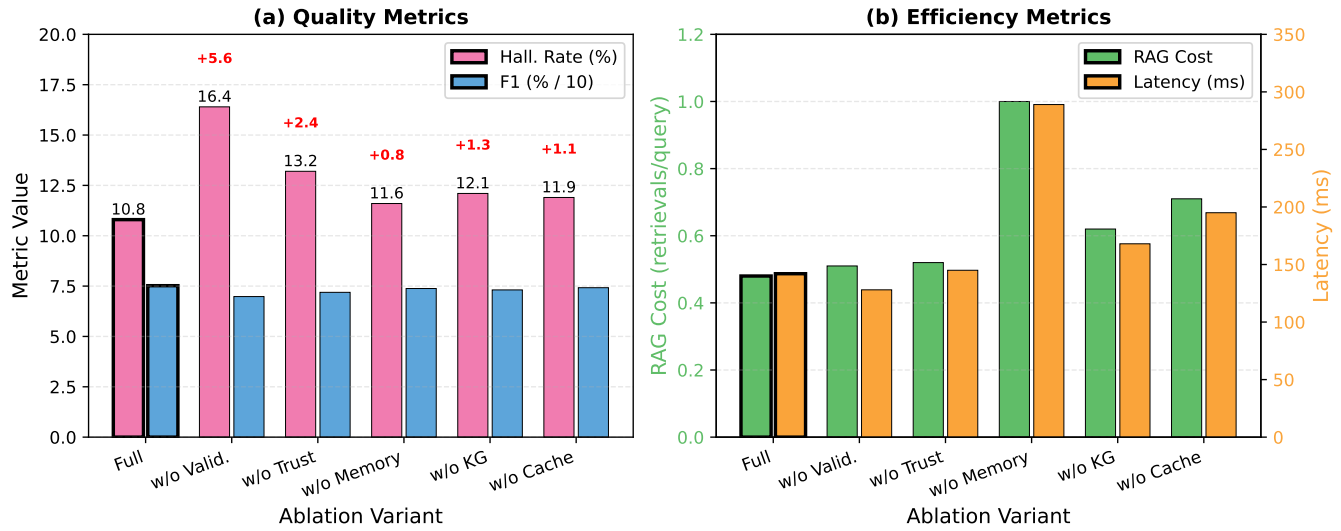


Figure 3: Ablation study on TempQuestions. (a) Removing validation causes the largest quality degradation (+5.6pp hallucination rate). (b) Removing memory doubles RAG cost and latency.

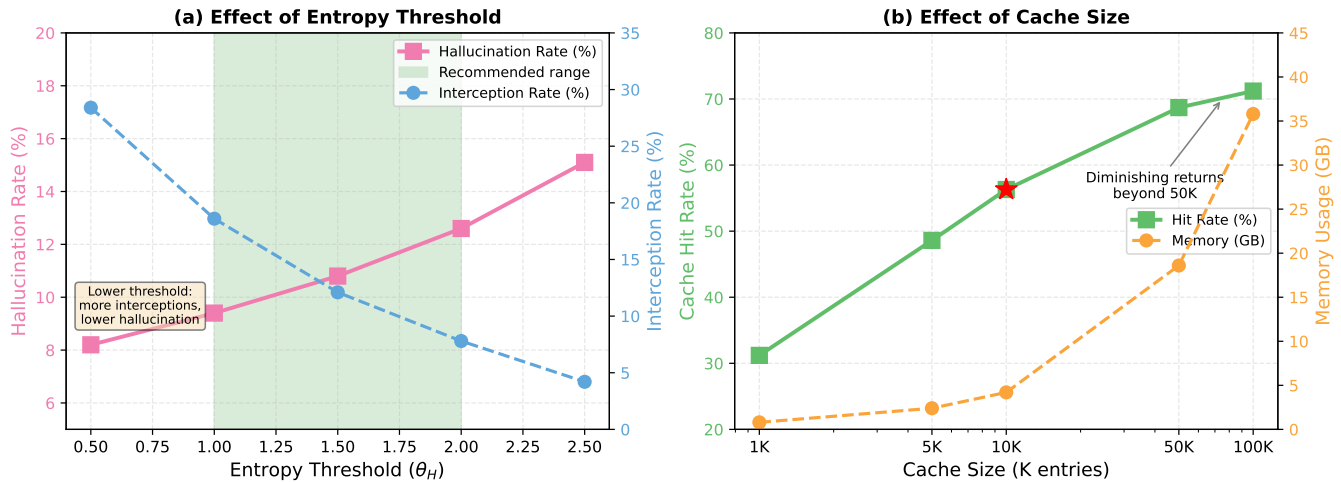


Figure 4: Sensitivity analysis. (a) Lower entropy thresholds reduce hallucinations but increase latency. (b) Cache hit rate improves with size but shows diminishing returns.

- [7] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. 2024. Detecting hallucinations in large language models using semantic entropy. *Nature* 630, 8017 (2024), 625–630.
- [8] Ke Hong, Xiuhong Li, Lufang Chen, Qiuli Mao, Xuefei Ning, Guohao Dai, Shengen Yan, Yun Liang, and Yu Wang. 2025. SOLA: Optimizing SLO Attainment for Large Language Model Serving with State-Aware Scheduling. In *Proceedings of Machine Learning and Systems*, Vol. 7.
- [9] Jeongyeon Hwang, Junyoung Park, Hyejin Park, Sangdon Park, and Jungseul Ok. 2024. Retrieval-Augmented Generation with Estimation of Source Reliability. *arXiv preprint arXiv:2410.22954* (2024).
- [10] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. 2024. RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation. *arXiv preprint arXiv:2404.12457* (2024).
- [11] Jannik Kossen, Sebastian Farquhar, Yarin Gal, and Tom Rainforth. 2024. Semantic Entropy Probes: Robust and Cheap Hallucination Detection in LLMs. *arXiv preprint arXiv:2406.15927* (2024).
- [12] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural Questions: A Benchmark for Question Answering Research. *Transactions of the Association for Computational Linguistics* 7 (2019), 452–466.
- [13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. ACM, 611–626.
- [14] Yassir Lairgi, Ludovic Moncla, Rémy Cazabet, Khalid Benabdeslem, and Pierre Cléau. 2024. iText2KG: Incremental Knowledge Graphs Construction Using Large Language Models. In *Proceedings of the 25th International Conference on Web Information Systems Engineering (WISE)*. Springer, 260–275.
- [15] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel,

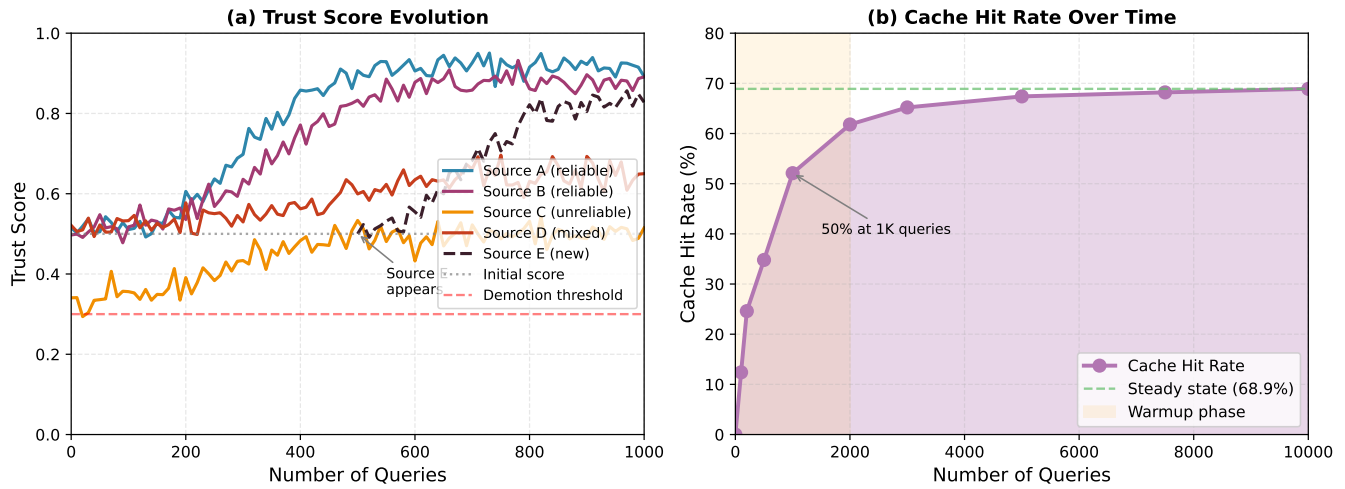


Figure 5: System dynamics. (a) Trust scores converge based on source reliability. (b) Cache hit rate stabilizes at 69% after warmup.

- Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., 9459–9474.
- [16] Meta AI. 2024. Llama 3 Model Card. *GitHub* (2024). <https://github.com/meta-llama/llama3>.
- [17] Juntong Song, Xingguang Wang, Juno Zhu, Yuanhao Wu, Adam Kovacs, and Gábor Recski. 2024. RAG-HAT: A Hallucination-Aware Tuning Pipeline for LLM in Retrieval-Augmented Generation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*. Association for Computational Linguistics, 1519–1530.
- [18] Zhongxiang Sun, Xiaoxue Zang, Kai Zheng, Jun Xu, Xiao Zhang, Weijie Yu, Yang Song, and Han Li. 2024. ReDeEP: Detecting Hallucination in Retrieval-Augmented Generation via Mechanistic Interpretability. *arXiv preprint arXiv:2410.11414* (2024).
- [19] Shuhe Wang, Xiaofei Sun, Xiaoya Li, Rongbin Ouyang, Fei Wu, Tianwei Zhang, Jiwei Li, and Guoyin Wang. 2025. GPT-NER: Named Entity Recognition via Large Language Models. In *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics, 3988–4003.
- [20] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. 2024. Corrective Retrieval Augmented Generation. *arXiv preprint arXiv:2401.15884* (2024).
- [21] Xiao Yang, Kai Chen, Zhiwei Sun, Tianyu Feng, Zhao Xu, Yun Zhu, Mingxin Zhuang, Xin Li, Bingxuan Shao, Hui Zhou, Wenguang Gao, Bin Tan, Shengyang Liu, Shiyao Wang, Wei Jin, Yanan Wang, Zhiyu Li, Qingsong Li, Yong Liu, Linxu Jiang, Xiaoyu Xu, Yuan Zhu, Weipeng Wang, Zeyu Zeng, Fuzheng Chen, Yu Yang, Di Dong, and Yulong Li. 2024. CRAG – Comprehensive RAG Benchmark. *arXiv preprint arXiv:2406.04744* (2024).
- [22] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2369–2380.